

2 Un curso acelerado de Python

La gente sigue loca por Python tras veinticinco años, cosa que me resulta difícil de creer.

—Michael Palin

Todos los empleados nuevos de DataSciencester tienen que pasar obligadamente por orientación al empleado, cuya parte más interesante es un curso acelerado de Python.

No se trata de un tutorial extenso, sino que está destinado a destacar las partes del lenguaje que serán más importantes para nosotros (algunas de las cuales no suelen ser el objetivo de los tutoriales de Python habituales). Si nunca había utilizado Python antes, probablemente quiera complementar esto con algún tipo de tutorial para principiante.

El zen de Python

Python tiene una descripción un poco zen de sus principios de diseño,¹ que se pueden encontrar dentro del propio intérprete de Python escribiendo `import this` (importar esto). Uno de los más discutidos es:

There should be one—and preferably only one—obvious way to do it.

(Solo debería haber una, y preferiblemente solo una, forma obvia de hacerlo).

El código escrito de acuerdo con esta forma “obvia” (que no tiene por qué serlo en absoluto para un principiante) se describe a menudo como “pythonic” (emplearemos la traducción “pitónico” en castellano, aunque suene un poco raro). Aunque este libro no trata de Python, de vez en cuando

contrastaremos formas pitónicas y no pitónicas de realizar las mismas cosas, y en general tenderemos más a emplear las soluciones pitónicas a nuestros problemas.

Otros principios aluden a lo estético:

Beautiful is better than ugly. Explicit is better than implicit. Simple is better than complex.

(Lo bello es mejor que lo feo. Lo explícito es mejor que lo implícito. Lo simple es mejor que lo complejo).

Y representan ideales por los que lucharemos en nuestro código.

Conseguir Python

Nota: Como las instrucciones de instalación de las cosas pueden cambiar, mientras que los libros impresos no, se pueden encontrar instrucciones actualizadas para instalar Python en el repositorio GitHub del libro.² Conviene revisar las del sitio web si las incluidas aquí no funcionan.

Se puede descargar Python desde Python.org.³ Pero, si todavía no se dispone de él, es más recomendable instalar la distribución Anaconda,⁴ que ya incluye la mayoría de las librerías necesarias para hacer ciencia de datos.

Cuando escribí la primera versión de este volumen, Python 2.7 seguía siendo el preferido por la mayoría de los científicos de datos. Por eso la primera edición del libro estaba basada en esta versión.

Pero, en los últimos años, casi todo el mundo ha migrado a Python 3. Las últimas versiones de Python tienen muchas funciones que permiten escribir código limpio con mayor facilidad, y nos aprovecharemos de otras que solo están disponibles en la versión 3.6 de Python o posterior, lo que significa que habría que conseguir esta versión más reciente de Python (además, muchas librerías útiles ya no soportan Python 2.7; otra razón más para cambiar).

Entornos virtuales

Ya desde el próximo capítulo utilizaremos la librería matplotlib para generar gráficas y diagramas o tablas. Esta librería no forma parte de Python; hay que instalarla por separado. Todos los proyectos de ciencia de datos que realicemos requerirán alguna combinación de librerías externas, a veces con versiones específicas que difieren de las empleadas en otros proyectos. Si tuviéramos una única instalación de Python, estas librerías entrarían en conflicto y provocarían todo tipo de problemas.

La solución estándar es utilizar entornos virtuales, es decir, entornos aislados de Python que mantienen sus propias versiones de librerías del lenguaje (y, dependiendo del modo en que se configure el entorno, del lenguaje en sí mismo).

Recomiendo instalar la distribución de Python denominada Anaconda, de modo que en esta sección explicaré cómo funcionan los entornos de Anaconda. También se puede utilizar el módulo venv integrado⁵ o instalar virtualenv,⁶ en cuyo caso habría que seguir sus instrucciones.

Para crear un entorno virtual (Anaconda) basta con hacer lo siguiente:

```
# crear un entorno Python 3.6 llamado "dsfs"
conda create -n dsfs python=3.6
```

Siguiendo los mensajes, logramos un entorno virtual llamado “dsfs”, con estas instrucciones:

```
#
# Para activar este entorno utilice:
# > source activate dsfs
#
# Para desactivar un entorno activo utilice:
# > source deactivate
#
```

Como se indica, el entorno se activa utilizando:

```
source activate dsfs
```



```
print(i)                                # última línea del bloque "for i"
print("done looping")
```

Así, el código de Python resulta fácilmente legible, pero también significa que hay que tener mucho cuidado con el formato.

Advertencia: Los programadores suelen debatir sobre si utilizar tabuladores o espacios para la sangría. En muchos lenguajes eso no importa, pero Python considera los tabuladores y los espacios como distintos tipos de sangría y el código no se ejecutará si se mezclan los dos. Al escribir en Python siempre hay que utilizar espacios, nunca tabuladores (si se escribe código en un editor es posible configurarlo de forma que la tecla **Tab** inserte espacios).

Los espacios en blanco se ignoran dentro de paréntesis y corchetes, lo que puede resultar útil para cálculos largos:

```
long_winded_computation    (1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10 + 11 + 12
=                            +
                            13 + 14 + 15 + 16 + 17 + 18 + 19 + 20)
```

Y para que el código resulte más fácil de leer:

```
list_of_lists = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
easier_to_read_list_of_lists =      [[1, 2, 3],
                                      [4, 5, 6],
                                      [7, 8, 9]]
```

También se puede utilizar una barra invertida para indicar que una sentencia continúa en la siguiente línea, aunque pocas veces se hace:

```
two_plus_three = 2 + \
                 3
```

Una consecuencia del formato con espacios en blanco es que puede ser difícil copiar y pegar código en el intérprete de Python. Por ejemplo, al intentar pegar este código:

```
for i in [1, 2, 3, 4, 5]:  
    # observe la línea en blanco  
    print(i)
```

En el *shell* normal de Python, aparecería este error:

```
IndentationError: expected an indented block
```

Porque el intérprete piensa que la línea vacía señala el final del bloque del bucle `for`.

IPython tiene una función mágica llamada `%paste` que pega correctamente lo que haya en el portapapeles, con espacios en blanco y todo. Solamente esto ya es una muy buena razón para utilizar IPython.

Módulos

Ciertas funciones de Python no se cargan por defecto. Entre ellas, hay funciones que están incluidas como parte del lenguaje y funciones externas que cada usuario puede descargar por su cuenta. Para poder utilizar estas funciones es necesario importar los módulos que las contienen.

Una forma de hacer esto es simplemente importando el propio módulo:

```
import re  
my_regex = re.compile("[0-9]+", re.I)
```

Aquí, `re` es el módulo que contiene funciones y constantes para trabajar con expresiones regulares. Tras este tipo de `import` hay que poner delante de esas funciones el prefijo `re.` para poder acceder a ellas.

Si ya había un `re` distinto en el código que se está utilizando, se puede emplear otro nombre:

```
import re as regex  
my_regex = regex.compile("[0-9]+", regex.I)
```

También se podría hacer esto si el módulo tiene un nombre poco manejable o si se va a escribir muy a menudo. Por ejemplo, un convenio estándar cuando se visualizan datos con `matplotlib` es:

```
import matplotlib.pyplot as plt
plt.plot(...)
```

Si se necesitan algunos valores específicos de un módulo, se pueden importar de manera explícita y utilizarlos sin reservas:

```
from collections import defaultdict, Counter
lookup = defaultdict(int)
my_counter = Counter()
```

Siendo malas personas, podríamos importar el contenido completo de un módulo en nuestro espacio de nombres, lo que podría sobrescribir involuntariamente variables que ya estaban definidas:

```
match = 10
from re import *      # oh, oh, re tiene una función que se llama igual
print(match)          # "<function match at 0x10281e6a8>"
```

Pero, como en realidad no somos malas personas, nunca haremos esto.

Funciones

Una función es una regla para tomar cero o más entradas y devolver una salida correspondiente. En Python, las funciones se definen normalmente utilizando `def`:

```
def double(x):
    """
    This is where you put an optional docstring that explains what the
    function does. For example, this function multiplies its input by 2.
    """
    return x * 2
```

Las funciones de Python son de primera clase, lo que significa que podemos asignarlas a variables y pasarlas a funciones como si se tratara de cualesquiera otros argumentos:

```
def apply_to_one(f):
```

```

    """Calls the function f with 1 as its argument"""
    return f(1)
my_double = double                # se refiere a la función anteriormente
                                  # definida
x =                               # es igual a 2
apply_to_one(my_double)

```

También es fácil crear funciones anónimas cortas, denominadas lambdas:

```

y = apply_to_one(lambda x: x + 4)    # es igual a 5

```

Se pueden asignar lambdas a variables, aunque la mayoría de la gente dirá que en su lugar solo hay que utilizar def:

```

another_double = lambda x: 2 * x      # no haga esto
def another_double(x):
    """Do this instead"""
    return 2 * x

```

A los parámetros de función también se les pueden asignar argumentos predeterminados, que solamente tienen que especificarse cuando se desea un valor distinto al predeterminado:

```

def my_print(message = "my default message"):
    print(message)
my_print("hello")           # imprime 'hello'
my_print()                  # imprime 'my default message'

```

Algunas veces es útil especificar argumentos por el nombre:

```

def full_name(first = "What's-his-name", last = "Something"):
    return first + " " + last
full_name("Joel", "Grus")      # "Joel Grus"
full_name("Joel")              # "Joel Something"
full_name(last="Grus")         # "What's-his-name Grus"

```

Vamos a crear muchas muchas funciones.

Cadenas

Las cadenas (o *strings*) pueden estar delimitadas por comillas simples o dobles (pero las comillas tienen que ir en pares):

```
single_quoted_string = 'data science'
double_quoted_string = "data science"
```

Python utiliza las barras invertidas para codificar caracteres especiales. Por ejemplo:

```
tab_string = "\t"      # representa el carácter del tabulador
len(tab_string)        # es 1
```

Si queremos barras invertidas como tales (las que se utilizan en nombres de directorio de Windows o en expresiones regulares), se pueden crear cadenas en bruto (*raw strings*) utilizando `r""`:

```
not_tab_string = r"\t"    # representa los caracteres '\' y 't'
len(not_tab_string)       # es 2
```

Se pueden crear cadenas multilínea utilizando comillas triples:

```
multi_line_string = """This is the first line,
and this is the second line
and this is the third line."""
```

Una función nueva en Python es la *f-string*, que ofrece una sencilla manera de sustituir valores por cadenas. Por ejemplo, si nos dieran el nombre y el apellido por separado:

```
first_name = "Joel"
last_name = "Grus"
```

Querríamos combinarlos como un nombre completo. Hay distintas formas de construir una cadena `full_name`:

```
full_name1 = first_name + " " + last_name      # suma de cadenas
full_name2 = "{0} {1}".format(first_name, last_name)  # string.format
```

Pero el método *f-string* es mucho más manejable:

```
full_name3 = f"{first_name} {last_name}"
```

Y lo preferiremos a lo largo del libro.

Excepciones

Cuando algo va mal, Python levanta una excepción. Si no se controlan adecuadamente, las excepciones pueden hacer que el programa se cuelgue. Se pueden manejar utilizando `try` y `except`:

```
try:
    print(0 / 0)
except ZeroDivisionError:
    print("cannot divide by zero")
```

Aunque en muchos lenguajes las excepciones no están bien consideradas, en Python no hay problema en utilizarlas para que el código sea más limpio, así que en ocasiones lo haremos.

Listas

Probablemente la estructura de datos más esencial de Python es la lista, que no es más que una colección ordenada (similar a lo que en otros lenguajes se podría denominar *array*, pero con funcionalidad añadida):

```
integer_list = [1, 2, 3]
heterogeneous_list = ["string", 0.1, True]
list_of_lists = [integer_list, heterogeneous_list, []]
list_length = len(integer_list)           # es igual a 3
list_sum = sum(integer_list)              # es igual a 6
```

Se puede obtener o establecer el elemento *n* de una lista con corchetes:

```
x = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
zero = x[0]           # es igual a 0, las listas están indexadas al 0
```

```

one = x[1]          # es igual a 1
nine = x[-1]        # es igual a 9, 'pitónico' para el último elemento
eight = x[-2]       # es igual a 8, 'pitónico' para el penúltimo elemento
x[0] = -1           # ahora x es [-1, 1, 2, 3, ..., 9]

```

También se pueden utilizar corchetes para crear *slices* en listas (cortes o arreglos). El corte `i:j` significa todos los elementos desde `i` (incluido) hasta `j` (excluido). Si dejamos fuera el principio del corte, lo extraeremos desde el principio de la lista, pero, si dejamos fuera el final del corte, lo extraeremos hasta el final:

```

first_three = x[:3]          # [-1, 1, 2]
three_to_end = x[3:]        # [3, 4, ..., 9]
one_to_four = x[1:5]        # [1, 2, 3, 4]
last_three = x[-3:]         # [7, 8, 9]
without_first_and_last = x[1:-1] # [1, 2, ..., 8]
copy_of_x = x[:]            # [-1, 1, 2, ..., 9]

```

De forma similar se pueden crear cortes de cadenas y otros tipos “secuenciales”.

Un corte puede admitir un tercer argumento para indicar su *stride* (avance), que puede ser negativo:

```

every_third = x[::3]        # [-1, 3, 6, 9]
five_to_three = x[5:2:-1]   # [5, 4, 3]

```

Python tiene un operador `in` para comprobar los miembros de la lista:

```

1 in [1, 2, 3]      # True
0 in [1, 2, 3]      # False

```

Esta comprobación implica examinar los elementos de la lista de uno en uno, lo que significa que probablemente no se debería utilizar a menos que se sepa que la lista es pequeña (o a menos que no nos preocupe el tiempo que tarde en hacerse la comprobación).

Es fácil concatenar listas. Si se desea modificar una lista en su lugar, se puede utilizar `extend` para añadir elementos de otra colección:

```
x = [1, 2, 3]
x.extend([4, 5, 6])      # x es ahora [1, 2, 3, 4, 5, 6]
```

Si no queremos modificar x, podemos ampliar la lista:

```
x = [1, 2, 3]
y = x + [4, 5, 6]      # y es [1, 2, 3, 4, 5, 6]; x no ha cambiado
```

Lo más frecuente que haremos será añadir elementos a listas uno a uno:

```
x = [1, 2, 3]
x.append(0)             # x es ahora [1, 2, 3, 0]
y = x[-1]              # es igual a 0
z = len(x)              # es igual a 4
```

A menudo, es conveniente desempaquetar listas cuando se sabe cuántos elementos contienen:

```
x, y = [1, 2]          # ahora x es 1, y es 2
```

Aunque obtendremos un `ValueError` si no tenemos el mismo número de elementos en ambos lados.

Algo que se utiliza habitualmente es un carácter de subrayado para un valor del que nos vamos a deshacer:

```
_, y = [1, 2]          # ahora y == 2, no nos importa el primer elemento
```

Tuplas

Las tuplas son las primas inmutables de las listas. Casi todo lo que se le puede hacer a una lista que implique modificarla, se le puede hacer a una tupla. Se especifica una tupla utilizando paréntesis (o nada) en lugar de corchetes:

```
my_list = [1, 2]
my_tuple = (1, 2)
```

```

other_tuple = 3, 4
my_list[1] = 3      # my_list es ahora [1, 3]
try:
    my_tuple[1] = 3
except TypeError:
    print("cannot modify a tuple")

```

Las tuplas son una forma cómoda de devolver varios valores de funciones:

```

def sum_and_product(x, y):
    return (x + y), (x * y)
sp = sum_and_product(2, 3)      # sp es (5, 6)
s, p = sum_and_product(5, 10)  # s es 15, p es 50

```

Las tuplas (y las listas) se pueden utilizar para asignación múltiple:

```

x, y = 1,      # ahora x es 1, y es 2
2
x, y = y,      # Forma pitónica de intercambiar variables; ahora x es 2, y es
x              1

```

Diccionarios

Otra estructura de datos fundamental es un diccionario, que asocia valores a claves y permite recuperar rápidamente el valor correspondiente a una determinada clave:

```

empty_dict = {}          # pitónico
empty_dict2 = dict()     # menos pitónico
grades = {"Joel": 80, "Tim": 95}  # dict literal

```

Se puede consultar el valor para una clave utilizando corchetes:

```

joels_grade = grades["Joel"]      # es igual a 80

```

Pero se obtendrá un `KeyError` si se pregunta por una clave que no está en el diccionario:

```
try:
    kates_grade = grades["Kate"]
except KeyError:
    print("no grade for Kate!")
```

Se puede comprobar la existencia de una clave utilizando in:

```
joel_has_grade = "Joel" in grades      # True
kate_has_grade = "Kate" in grades      # False
```

Esta verificación de membresía es aún más rápida para diccionarios grandes.

Los diccionarios tienen un método get que devuelve un valor predeterminado (en lugar de levantar una excepción) cuando se consulta una clave que no está en el diccionario:

```
joels_grade = grades.get("Joel", 0)    # es igual a 80
kates_grade = grades.get("Kate", 0)    # es igual a 0
no_ones_grade = grades.get("No One")   # el valor predeterminado es None
```

Se pueden asignar pares clave/valor utilizando los mismos corchetes:

```
grades["Tim"] = 99                     # reemplaza el valor anterior
grades["Kate"] = 100                   # añade una tercera entrada
num_students = len(grades)            # es igual a 3
```

Como vimos en el capítulo 1, se pueden utilizar diccionarios para representar datos estructurados:

```
tweet = {
    "user" : "joelgrus",
    "text" : "Data Science is Awesome",
    "retweet_count" : 100,
    "hashtags" : ["#data", "#science", "#datascience", "#awesome", "#yolo"]
}
```

Aunque pronto veremos un enfoque mejor.

Además de buscar claves específicas, también podemos mirarlas todas:

```

tweet_keys = tweet.keys()      # iterable para las claves
tweet_values =                 # iterable para los valores
tweet.values()
tweet_items =                  # iterable para las tuplas (clave, valor)
tweet.items()
"user" in tweet_keys           # True, pero no pitónico
"user" in tweet                # forma pitónica de comprobar claves
"joelgrus" in tweet_values     # True (es lenta, pero la única forma de
                               verificar)

```

Las claves de diccionario pueden ser “*hashables*”; en particular, no se pueden utilizar listas como claves. Si se necesita una clave multiparte, probablemente se debería utilizar una tupla o idear un modo de convertir la clave en una cadena.

defaultdict

Imaginemos que estamos intentando contar las palabras de un documento. Un método obvio para lograrlo es crear un diccionario en el que las claves sean palabras y los valores sean contadores. Al comprobar cada palabra, se puede incrementar su contador si ya está en el diccionario y añadirlo al diccionario si no estaba:

```

word_counts = {}
for word in document:
    if word in word_counts:
        word_counts[word] += 1
    else:
        word_counts[word] = 1

```

Se podría utilizar también el sistema “mejor pedir perdón que permiso” y simplemente manejar la excepción al intentar consultar una clave inexistente:

```

word_counts = {}
for word in document:
    try:
        word_counts[word] += 1
    except KeyError:
        word_counts[word] = 1

```

Un tercer enfoque es utilizar `get`, que se comporta con mucha elegancia con las claves inexistentes:

```
word_counts = {}
for word in document:
    previous_count = word_counts.get(word, 0)
    word_counts[word] = previous_count + 1
```

Todo esto es muy poco manejable, razón por la cual `defaultdict` es útil. Un `defaultdict` es como un diccionario normal, excepto que, cuando se intenta buscar una clave que no contiene, primero añade un valor para ella utilizando una función de argumento cero suministrada al crearla. Para utilizar diccionarios `defaultdicts`, es necesario importarlos de `collections`:

```
from collections import defaultdict
word_counts = defaultdict(int)      # int() produce 0
for word in document:
    word_counts[word] += 1
```

También pueden resultar útiles con `list` o `dict`, o incluso con nuestras propias funciones:

```
dd_list = defaultdict(list)          # list() produce una lista vacía
dd_list[2].append(1)                 # ahora dd_list contiene {2: [1]}
dd_dict = defaultdict(dict)          # dict() produce un dict vacío
dd_dict["Joel"]["City"] = "Seattle"  # {"Joel" : {"City": "Seattle"}}
dd_pair = defaultdict(lambda: [0, 0])
dd_pair[2][1] = 1                    # ahora dd_pair contiene {2: [0, 1]}
```

Serán útiles cuando estemos utilizando diccionarios para “recopilar” resultados según alguna clave y no queramos comprobar todo el tiempo si la clave sigue existiendo.

Contadores

Un `Counter` convierte una secuencia de valores en un objeto de tipo

`defaultdict(int)` mapeando claves en contadores:

```
from collections import Counter
c = Counter([0, 1, 2, 0])      # c es (básicamente) {0: 2, 1: 1, 2: 1}
```

Lo que nos ofrece un modo muy sencillo de resolver problemas de `word_counts`:

```
# recuerde, document es una lista de palabras
word_counts = Counter(document)
```

Una instancia `Counter` tiene un método `most_common` que se utiliza con frecuencia:

```
# imprime las 10 palabras más comunes y sus contadores
for word, count in word_counts.most_common(10):
    print(word, count)
```

Conjuntos

Otra estructura de datos útil es el conjunto o *set*, que representa una colección de distintos elementos. Se pueden definir un conjunto listando sus elementos entre llaves:

```
primes_below_10 = {2, 3, 5, 7}
```

Sin embargo, esto no funciona con conjuntos vacíos, dado que `{}` ya significa “dict vacío”. En ese caso, habrá que utilizar la propia `set()`:

```
s = set()
s.add(1)      # s es ahora {1}
s.add(2)      # s es ahora {1, 2}
s.add(2)      # s sigue siendo {1, 2}
x = len(s)    # es igual a 2
y = 2 in s    # es igual a True
z = 3 in s    # es igual a False
```

Utilizaremos conjuntos por dos razones principales. La primera es que `in`

es una operación muy rápida con conjuntos. Si tenemos una gran colección de elementos que queremos utilizar para hacer una prueba de membresía, un conjunto es más adecuado que una lista:

```
stopwords_list = ["a", "an", "at"] + hundreds_of_other_words + ["yet", "you"]
"zip" in stopwords_list      # False, pero hay que verificar cada elemento
stopwords_set = set(stopwords_list)
"zip" in stopwords_set      # muy rápido de comprobar
```

La segunda razón es encontrar los elementos distintos de una colección:

```
item_list = [1, 2, 3, 1, 2, 3]
num_items = len(item_list)      # 6
item_set = set(item_list)      # {1, 2, 3}
num_distinct_items = len(item_set) # 3
distinct_item_list = list(item_set) # [1, 2, 3]
```

Utilizaremos conjuntos con menos frecuencia que diccionarios y listas.

Flujo de control

Como en la mayoría de los lenguajes de programación, se puede realizar una acción de forma condicional utilizando `if`:

```
if 1 > 2:
    message = "if only 1 were greater than two..."
elif 1 > 3:
    message = "elif stands for 'else if'"
else:
    message = "when all else fails use else (if you want to)"
```

Se puede también escribir un ternario `if-then-else` en una sola línea, cosa que haremos muy de tanto en tanto:

```
parity = "even" if x % 2 == 0 else "odd"
```

Python tiene un bucle `while`:

```
x = 0
while x < 10:
    print(f"{x} is less than 10")
    x += 1
```

Aunque con mucha más frecuencia usaremos for e in:

```
# range(10) es los números 0, 1, ..., 9
for x in range(10):
    print(f"{x} is less than 10")
```

Si necesitáramos lógica más compleja, podríamos utilizar continue y break:

```
for x in range(10):
    if x == 3:
        continue      # va inmediatamente a la siguiente repetición
    if x == 5:
        break          # sale del todo del bucle
    print(x)
```

Esto imprimirá 0, 1, 2 y 4.

Verdadero o falso

Los valores booleanos en Python funcionan igual que en casi todos los demás lenguajes, excepto que llevan la primera letra en mayúscula:

```
one_is_less_than_two = 1 < 2          # es igual a True
true_equals_false = True == False     # es igual a False
```

Python utiliza el valor None para indicar un valor no existente. Es similar al null de otros lenguajes:

```
x = None
assert x == None, "this is the not the Pythonic way to check for None"
assert x is None, "this is the Pythonic way to check for None"
```

Python permite utilizar cualquier valor donde espera un booleano. Las siguientes expresiones son todas “falsas”:

- `False`.
- `None`.
- `[]` (una list vacía).
- `{}` (un dict vacío).
- `""`.
- `set()`.
- `0`.
- `0.0`.

Casi todo lo demás se trata como `True`. Ello permite utilizar fácilmente sentencias `if` para probar listas vacías, cadenas vacías, diccionarios vacíos, etc. También produce en ocasiones errores complicados si no se espera este comportamiento:

```
s = some_function_that_returns_a_string()
if s:
    first_char = s[0]
else:
    first_char = ""
```

Una forma más corta (pero posiblemente más confusa) de hacer lo mismo es:

```
first_char = s and s[0]
```

Ya que `and` devuelve su segundo valor cuando el primero es “verdadero”, y el primer valor cuando no lo es. De forma similar, si `x` es o bien un número o posiblemente `None`:

```
safe_x = x or 0
```

Es definitivamente un número, aunque:

```
safe_x = x if x is not None else 0
```

Es posiblemente más legible.

Python tiene una función `all`, que toma un iterable y devuelve `True` exactamente cuando cada elemento es verdadero, y una función `any`, que devuelve `True` cuando al menos un elemento es verdad:

```
all([True, 1, {3}])    # True, todos son verdaderos
all([True, 1, {}])     # False, {} is falso
any([True, 1, {}])     # True, True is verdadero
all([])                # True, no hay elementos falsos en la lista
any([])                # False, no hay elementos verdaderos en la lista
```

Ordenar

Toda lista de Python tiene un método `sort` que la ordena en su lugar. Si no queremos estropear nuestra lista, podemos usar la función `sorted`, que devuelve una lista nueva:

```
x = [4, 1, 2, 3]
y = sorted(x)          # y es [1, 2, 3, 4], x queda igual
x.sort()               # ahora x es [1, 2, 3, 4]
```

Por defecto, `sort` (y `sorted`) ordena una lista de menor a mayor basándose en comparar inocentemente los elementos uno con otro.

Si queremos que los elementos estén ordenados de mayor a menor, se puede especificar un parámetro `reverse=True`. Y, en lugar de comparar los elementos por sí mismos, se pueden comparar los resultados de una función que se especifica con `key`:

```
# ordena la lista por valor absoluto de mayor a menor
x = sorted([-4, 1, -2, 3], key=abs, reverse=True)    # is [-4, 3, -2, 1]
# ordena las palabras y contadores del contador mayor al menor
wc = sorted(word_counts.items(),
             key=lambda word_and_count: word_and_count[1],
             reverse=True)
```

Comprensiones de listas

Con frecuencia, vamos a querer transformar una lista en otra distinta seleccionando solo determinados elementos, transformando elementos o haciendo ambas cosas. La forma pitónica de hacer esto es con *list comprehensions*, o comprensiones de listas:

```
even_numbers = [x for x in range(5) if x % 2 == 0]      # [0, 2, 4]
squares = [x * x for x in range(5)]                  # [0, 1, 4, 9, 16]
even_squares = [x * x for x in even_numbers]          # [0, 4, 16]
```

De forma similar, se pueden convertir listas en diccionarios o conjuntos:

```
square_dict = {x: x * x for x in range(5)}           # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
square_set = {x * x for x in [1, -1]}                 # {1}
```

Si no necesitamos el valor de la lista, es habitual utilizar un guion bajo como variable:

```
zeros = [0 for _ in even_numbers]                   # tiene la misma longitud que even_numbers
```

Una comprensión de lista puede incluir varios for:

```
pairs = [(x, y)
          for x in range(10)
          for y in range(10)]                        # 100 pares (0,0) (0,1) ... (9,8), (9,9)
```

Y después los for pueden utilizar los resultados de los anteriores:

```
increasing_pairs = [(x, y)
                     for x in range(10)
                     for y in range(x + 1, 10)]      # solo pares con x < y, # range(bajo, alto) es igual a # [bajo, bajo + 1, ..., alto-1]
```

Utilizaremos mucho las comprensiones de listas.

Pruebas automatizadas y assert

Como científicos de datos, escribiremos mucho código. ¿Cómo podemos estar seguros de que nuestro código es correcto? Una forma es con tipos (de los que hablaremos en breve), pero otra forma es con *automated tests* o pruebas automatizadas.

Hay estructuras muy complicadas para escribir y ejecutar pruebas, pero en este libro nos limitaremos a utilizar sentencias `assert`, que harán que el código levante un `AssertionError` si la condición especificada no es verdadera:

```
assert 1 + 1 == 2
assert 1 + 1 == 2, "1 + 1 should equal 2 but didn't"
```

Como podemos ver en el segundo caso, se puede añadir si se desea un mensaje que se imprimirá si la evaluación falla.

No es especialmente interesante evaluar que $1 + 1 = 2$, pero lo es mucho más verificar que las funciones que escribamos hagan lo que se espera de ellas:

```
def smallest_item(xs):
    return min(xs)
assert smallest_item([10, 20, 5, 40]) == 5
assert smallest_item([1, 0, -1, 2]) == -1
```

A lo largo del libro utilizaremos `assert` de esta forma. Es una buena práctica, y yo animo a utilizar libremente esta sentencia (en el código del libro que encontramos en GitHub se comprueba que contiene muchas más sentencias `assert` de las que aparecen impresas en el libro, lo que me permite estar seguro de que el código que he escrito es correcto).

Otro uso menos habitual es evaluar cosas sobre entradas a funciones:

```
def smallest_item(xs):
    assert xs, "empty list has no smallest item"
    return min(xs)
```

Haremos esto de vez en cuando, pero será más frecuente utilizar `assert`

para verificar que el código escrito es correcto.

Programación orientada a objetos

Como muchos lenguajes, Python permite definir clases que encapsulan los datos y las funciones que operan con ellos. Las utilizaremos algunas veces para que nuestro código sea más limpio y sencillo. Probablemente, es más fácil explicarlas construyendo un ejemplo con muchas anotaciones.

Aquí vamos a crear una clase que represente un “contador de clics”, del tipo de los que se ponen en la puerta para controlar cuántas personas han acudido al encuentro “Temas avanzados sobre ciencia de datos”.

Mantiene un contador (`count`), se le puede hacer clic (`clicked`) para aumentar la cuenta, permite lectura de contador (`read_count`) y se puede reiniciar (`reset`) de vuelta a cero (en la vida real una de estas clases pasa de 9999 a 0000, pero no vamos a preocuparnos de eso ahora).

Para definir una clase, utilizamos la palabra clave `class` y un nombre de tipo PascalCase:

```
class CountingClicker:
    """A class can/should have a doctring, just like a function"""
```

Una clase contiene cero o más funciones miembro. Por convenio, cada una toma un primer parámetro, `self`, que se refiere a la instancia en particular de la clase.

Normalmente, una clase tiene un constructor, llamado `__init__`, que toma los parámetros que necesita para construir una instancia de dicha clase y hace cualquier configuración que se necesite:

```
def __init__(self, count = 0):
    self.count = count
```

Aunque el constructor tiene un nombre divertido, construimos las instancias del contador de clics utilizando solamente el nombre de la clase:

```

clicker1 = CountingClicker()           # inicializado a 0
clicker2 = CountingClicker(100)        # empieza con count=100
clicker3 = CountingClicker(count=100)  # forma más explícita de hacer lo mismo

```

Vemos que el nombre del método `__init__` empieza y termina con guiones bajos. A veces a estos métodos “mágicos” se les llama métodos “dunder” (término inventado que viene de *doubleUNDERscore*, es decir, doble guion bajo) y representan comportamientos “especiales”.

Nota: Los métodos de clase cuyos nombres empiezan con un guion bajo se consideran (por convenio) “privados”, y se supone que los usuarios de esa clase no les llaman directamente. Sin embargo, Python no impide a los usuarios llamarlos.

Otro método similar es `__repr__`, que produce la representación de cadena de una instancia de clase:

```

def __repr__(self):
    return f"CountingClicker(count={self.count})"

```

Y finalmente tenemos que implementar la API pública de la clase que hemos creado:

```

def click(self, num_times = 1):
    """Click the clicker some number ot times."""
    self.count += num_times
def read(self):
    return self.count
def reset(self):
    self.count = 0

```

Una vez definido, utilicemos `assert` para escribir algunos casos de prueba para nuestro contador de clics:

```

clicker = CountingClicker()
assert clicker.read() == 0, "clicker should start with count 0"
clicker.click()

```

```
clicker.click()
assert clicker.read() == 2, "after two clicks, clicker should have count 2"
clicker.reset()
assert clicker.read() == 0, "after reset, clicker should be back to 0"
```

Escribir pruebas como estas nos permite estar seguros de que nuestro código esté funcionando tal y como está diseñado, y que esto va a seguir siendo así siempre que le hagamos cambios.

También crearemos de vez en cuando subclases que heredan parte de su funcionalidad de una clase padre. Por ejemplo, podríamos crear un contador de clics no reinicializable utilizando `CountingClicker` como clase base y anulando el método `reset` para que no haga nada:

```
# Una subclase hereda todo el comportamiento de su clase padre.
class NoResetClicker(CountingClicker):
    # Esta clase tiene los mismos métodos que CountingClicker
    # Salvo que tiene un método reset que no hace nada.
    def reset(self):
        pass

clicker2 = NoResetClicker()
assert clicker2.read() == 0
clicker2.click()
assert clicker2.read() == 1
clicker2.reset()
assert clicker2.read() == 1, "reset shouldn't do anything"
```

Iterables y generadores

Una cosa buena de las listas es que se pueden recuperar determinados elementos por sus índices. Pero ¡esto no siempre es necesario! Una lista de mil millones de números ocupa mucha memoria. Si solo queremos los elementos uno cada vez, no hay una buena razón que nos haga conservarlos a todos. Si solamente terminamos necesitando los primeros elementos, generar los mil millones es algo tremendamente inútil.

A menudo, todo lo que necesitamos es pasar varias veces por la colección utilizando `for` e `in`. En este caso podemos crear generadores, que se pueden

iterar igual que si fueran listas, pero generan sus valores bajo petición.

Una forma de crear generadores es con funciones y con el operador yield:

```
def generate_range(n):
    i = 0
    while i < n:
        yield i      # cada llamada a yield produce un valor del generador
        i += 1
```

El siguiente bucle consumirá uno a uno los valores a los que se ha aplicado yield hasta que no quede ninguno:

```
for i in generate_range(10):
    print(f"i: {i}")
```

(En realidad, range es bastante perezosa de por sí, así que hacer esto no tiene ningún sentido).

Con un generador, incluso se puede crear una secuencia infinita:

```
def natural_numbers():
    """returns 1, 2, 3, ..."""
    n = 1
    while True:
        yield n
        n += 1
```

Aunque probablemente no deberíamos iterar sobre él sin utilizar algún tipo de lógica de interrupción.

Truco: La otra cara de la perezosa es que solo se puede iterar una única vez por un generador. Si hace falta pasar varias veces, habrá que volver a crear el generador cada vez o bien utilizar una lista. Si generar los valores resulta caro, podría ser una buena razón para utilizar una lista en su lugar.

Una segunda manera de crear generadores es utilizar las comprensiones envueltas en paréntesis:

```
evens_below_20 = (i for i in generate_range(20) if i % 2 == 0)
```

Una “comprensión de generador” como esta no hace nada hasta que se itera sobre ella (utilizando for o next). Podemos utilizar esto para crear complicadas líneas de proceso de datos:

```
# Ninguno de estos cálculos *hace* nada hasta que iteramos
data = natural_numbers()
evens = (x for x in data if x % 2 == 0)
even_squares = (x ** 2 for x in evens)
even_squares_ending_in_six = (x for x in even_squares if x % 10 == 6)
# y así sucesivamente
```

No pocas veces, cuando estemos iterando sobre una lista o un generador, no querremos solamente los valores, sino también sus índices. Para este caso habitual, Python ofrece una función enumerate, que convierte valores en pares (index, value):

```
names = ["Alice", "Bob", "Charlie", "Debbie"]
# no pitónico
for i in range(len(names)):
    print(f"name {i} is {names[i]}")
# tampoco pitónico
i = 0
for name in names:
    print(f"name {i} is {names[i]}")
    i += 1
# pitónico
for i, name in enumerate(names):
    print(f"name {i} is {name}")
```

Utilizaremos mucho esto.

Aleatoriedad

A medida que aprendamos ciencia de datos, necesitaremos con frecuencia generar números aleatorios, lo que podemos hacer con el módulo random:

```
import random
random.seed(10)      # esto asegura que obtenemos los mismos resultados cada vez
```

```
four_uniform_randoms = [random.random() for _ in range(4)]

# [0.5714025946899135,      # random.random() produce números
# 0.4288890546751146,      # de manera uniforme entre 0 y 1.
# 0.5780913011344704,      # Es la función random que utilizaremos
# 0.20609823213950174]     # con más frecuencia.
```

El módulo `random` produce en realidad números pseudoaleatorios (es decir, deterministas) basados en un estado interno que se puede configurar con `random.seed` si lo que se desea es obtener resultados reproducibles:

```
random.seed(10)           # establece la semilla en 10
print(random.random())    # 0.57140259469
random.seed(10)           # reinicia la semilla en 10
print(random.random())    # 0.57140259469 de nuevo
```

Algunas veces utilizaremos `random.randrange`, que toma uno o dos argumentos y devuelve un elemento elegido aleatoriamente del range correspondiente:

```
random.randrange(10)      # selecciona aleatoriamente de range(10) = [0, 1,
                          # ..., 9]
random.randrange(3,      # selecciona aleatoriamente de range(3, 6) = [3, 4,
6)                       # 5]
```

Hay varios métodos más que en ocasiones nos resultarán convenientes. Por ejemplo, `random.shuffle` reordena aleatoriamente los elementos de una lista:

```
up_to_ten = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
random.shuffle(up_to_ten)
print(up_to_ten)
# [7, 2, 6, 8, 9, 4, 10, 1, 3,      (sus resultados serán probablemente
5]                                  diferentes)
```

Si se necesita elegir aleatoriamente un elemento de una lista, se puede utilizar `random.choice`:

```
my_best_friend = random.choice(["Alice", "Bob", "Charlie"])    # "Bob" para mí
```

Y, si lo que hace falta es elegir aleatoriamente una muestra de elementos sin sustituto (es decir, sin duplicados), se puede utilizar `random.sample`:

```
lottery_numbers = range(60)
winning_numbers = random.sample(lottery_numbers,      # [16, 36, 10, 6, 25,
6)                                                    9]
```

Para elegir una muestra de elementos con sustituto (es decir, que permita duplicados), simplemente basta con hacer varias llamadas a `random.choice`:

```
four_with_replacement = [random.choice(range(10)) for _ in range(4)]
print(four_with_replacement)          # [9, 4, 4, 2]
```

Expresiones regulares

Las expresiones regulares ofrecen un modo de buscar texto. Son increíblemente útiles, pero también bastante complicadas (tanto, que hay escritos libros enteros sobre ellas). Entraremos en detalle las pocas veces que nos las encontremos; estos son algunos ejemplos de cómo utilizarlas en Python:

```
import re
re_examples = [
    not re.match("a", "cat"),          # Todas son True, porque
    re.search("a", "cat"),             # 'cat' no empieza por 'a'
    not re.search("c", "dog"),         # 'cat' contiene una 'a'
    3 == len(re.split("[ab]",         # 'dog' no contiene una 'c'.
    "carbs")),                        # Partido en a o b para
    "R-D-" == re.sub("[0-9]", "-",    ['c', 'r', 's'].
    "R2D2"),                          # Reemplaza dígitos por guiones.
]
assert all(re_examples), "all the regex examples should be True"
```

Algo importante a tener en cuenta es que `re.match` comprueba si el principio de una cadena coincide con una expresión regular, mientras que `re.search` lo comprueba con alguna parte de una cadena. En algún momento es probable que se mezclen los dos y creen problemas.

La documentación oficial en <https://docs.python.org/es/3/library/re.html> ofrece muchos más detalles.

Programación funcional

Nota: La primera edición de este libro presentaba en este momento las funciones de Python `partial`, `map`, `reduce` y `filter`. En mi viaje hacia la iluminación, me he dado cuenta de que es mejor evitarlas, y sus usos en el libro han sido reemplazados por comprensiones de listas, bucles `for` y otras construcciones más pitónicas.

Empaquetado y desempaquetado de argumentos

A menudo, necesitaremos empaquetar (*zip*) dos o más iterables juntos. La función `zip` transforma varios iterables en uno solo de tuplas de función correspondiente:

```
list1 = ['a', 'b', 'c']
list2 = [1, 2, 3]
# zip es perezoso, de modo que hay que hacer algo como lo siguiente
[pair for pair in zip(list1, list2)]    # es [('a', 1), ('b', 2), ('c', 3)]
```

Si las listas tienen distintas longitudes, `zip` se detiene tan pronto como termina la primera lista.

También se puede “desempaquetar” una lista utilizando un extraño truco:

```
pairs = [('a', 1), ('b', 2), ('c', 3)]
letters, numbers = zip(*pairs)
```

El asterisco (*) realiza desempaquetado de argumento, que utiliza los elementos de `pairs` como argumentos individuales para `zip`. Termina igual que si lo hubiéramos llamado:

```
letters, numbers = zip(('a', 1), ('b', 2), ('c', 3))
```

Se puede utilizar desempaqueado de argumento con cualquier función:

```
def add(a, b): return a + b
add(1, 2)          # devuelve 3
try:
    add([1, 2])
except TypeError:
    print("add expects two inputs")
add(*[1, 2])       # devuelve 3
```

Es raro que encontremos esto útil, pero, cuando lo hacemos, es un truco genial.

args y kwargs

Digamos que queremos crear una función de máximo orden que requiere como entrada una función *f* y devuelve una función nueva que para cualquier entrada devuelve el doble del valor de *f*:

```
def doubler(f):
    # Aquí definimos una nueva función que mantiene una referencia a f
    def g(x):
        return 2 * f(x)
    # Y devuelve esa nueva función
    return g
```

Esto funciona en algunos casos:

```
def f1(x):
    return x + 1
g = doubler(f1)
assert g(3) == 8, "(3 + 1) * 2 should equal 8"
assert g(-1) == 0, "(-1 + 1) * 2 should equal 0"
```

Sin embargo, no sirve con funciones que requieren algo más que un solo argumento:

```
def f2(x, y):
```

```

    return x + y
g = doubler(f2)
try:
    g(1, 2)
except TypeError:
    print("cas defined, g only takes one argument")

```

Lo que necesitamos es una forma de especificar una función que tome argumentos arbitrarios. Podemos hacerlo con desempaqueado de argumento y un poco de magia:

```

def magic(*args, **kwargs):
    print("unnamed args:", args)
    print("keyword args:", kwargs)
magic(1, 2, key="word", key2="word2")
# imprime
# argumentos sin nombre: (1, 2)
# argumentos de palabra clave: {'key': 'word', 'key2': 'word2'}

```

Es decir, cuando definimos una función como esta, args es una tupla de sus argumentos sin nombre y kwargs es un dict de sus argumentos con nombre. Funciona también a la inversa, si queremos utilizar una list (o tuple) y dict para proporcionar argumentos a una función:

```

def other_way_magic(x, y, z):
    return x + y + z
x_y_list = [1, 2]
z_dict = {"z": 3}
assert other_way_magic(*x_y_list, **z_dict) == 6, "1 + 2 + 3 should be 6"

```

Se podría hacer todo tipo de trucos extraños con esto; solo lo utilizaremos para producir funciones de máximo orden cuyas entradas puedan aceptar argumentos arbitrarios:

```

def doubler_correct(f):
    """works no matter what kind of inputs f expects"""
    def g(*args, **kwargs):
        """whatever arguments g is supplied, pass them through to f"""
        return 2 * f(*args, **kwargs)
    return g

```

```
g = doubler_correct(f2)
assert g(1, 2) == 6, "doubler should work now"
```

Como regla general, el código que escribamos será más correcto y legible si somos explícitos en lo que se refiere a los tipos de argumentos que las funciones que usemos requieren; de ahí que vayamos a utilizar args y kwargs solo cuando no tengamos otra opción.

Anotaciones de tipos

Python es un lenguaje de tipos dinámicos. Esto significa que en general no le importan los tipos de objetos que utilicemos, siempre que lo hagamos de formas válidas:

```
def add(a, b):
    return a + b
assert add(10, 5) == 15, "+ is valid for numbers"
assert add([1, 2], [3]) == [1, 2, 3], "+ is valid for lists"
assert add("hi ", "there") == "hi there", "+ is valid for strings"
try:
    add(10, "five")
except TypeError:
    print("cannot add an int to a string")
```

Mientras que en un lenguaje de tipos estáticos nuestras funciones y objetos tendrían tipos específicos:

```
def add(a: int, b: int) -> int:
    return a + b
add(10, 5)                # le gustaría que esto fuera correcto
add("hi ", "there")       # le gustaría que esto no fuera correcto
```

En realidad, las versiones más recientes de Python tienen (más o menos) esta funcionalidad. ¡La versión anterior de add con las anotaciones de tipos int es válida en Python 3.6! Sin embargo, estas anotaciones de tipos no hacen realmente nada. Aún se puede utilizar la función anotada add para

añadir cadenas, y la llamada a `add(10, "five")` seguirá levantando exactamente el mismo `TypeError`.

Dicho esto, sigue habiendo (al menos) cuatro buenas razones para utilizar anotaciones de tipos en el código Python que escribamos:

- Los tipos son una forma importante de documentación. Esto es doblemente cierto en un libro que utiliza código para enseñar conceptos teóricos y matemáticos. Comparemos las siguientes dos líneas de función:

```
def dot_product(x, y): ...  
# aún no hemos definido Vector, pero imagínese que lo habíamos hecho  
def dot_product(x: Vector, y: Vector) -> float: ...
```

Encuentro el segundo extremadamente más informativo; espero que también se lo parezca (en este punto me he acostumbrado tanto a la determinación de tipos que ahora sin ello encuentro Python difícil de leer).

- Hay herramientas externas (siendo la más popular `mypy`) que leerán el código que escribamos, inspeccionarán las anotaciones de tipos y ofrecerán errores de tipos antes siquiera de ejecutar el código. Por ejemplo, si ejecutamos `mypy` en un archivo que contiene `add("hi", "there")`, avisaría de lo siguiente:

```
error: Argument 1 to "add" has incompatible type "str"; expected "int"
```

Al igual que la prueba `assert`, esta es una buena forma de encontrar errores en el código antes de ejecutarlo. La narración del libro no implicará tal comprobación de tipo; sin embargo, ejecutaré una en segundo plano, lo que me permitirá asegurarme de que el libro en sí es correcto.

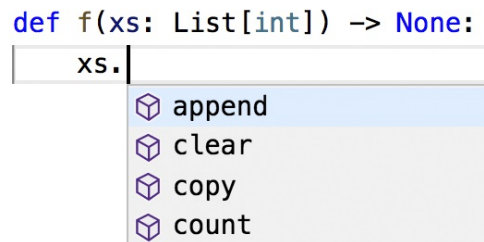
- Tener que pensar en los tipos de nuestro código nos obliga a diseñar funciones e interfaces más limpios:

```
from typing import Union  
def secretly_ugly_function(value, operation): ...
```

```
def ugly_function(value: int,  
                  operation: Union[str, int, float, bool]) -> int:  
    ...
```

Aquí tenemos una función cuyo parámetro de operación puede ser un string, un int, un float o un bool. Es muy probable que esta función sea frágil y difícil de utilizar, pero aún queda más claro cuando los tipos resultan explícitos. Hacer esto nos obligará a diseñar de un modo menos torpe, cosa que nuestros usuarios nos agradecerán.

- Utilizar tipos permite al editor que utilicemos ayudarnos con cosas como autocompletar (véase la figura 2.1) y enfadarnos por los errores de escritura.



```
def f(xs: List[int]) -> None:  
    xs.  
    append  
    clear  
    copy  
    count
```

A screenshot of the Visual Studio Code editor. It shows a function definition `def f(xs: List[int]) -> None:`. Below the opening curly brace, the text `xs.` is entered. A dropdown menu is visible, showing four list methods: `append`, `clear`, `copy`, and `count`. Each method is preceded by a small cube icon.

Figura 2.1. VSCode, pero probablemente otro editor haga lo mismo.

A veces, la gente insiste en que las comprobaciones de tipo pueden ser valiosas en proyectos grandes, pero no merecen la pena en otros más pequeños. No obstante, como casi no se tardan nada en escribir y permiten al editor ahorrarnos tiempo, yo mantengo que de verdad permiten escribir código con mayor rapidez, incluso en proyectos pequeños.

Por todas estas razones, todo el código del resto de este libro utilizará anotaciones de tipos. Supongo que algunos lectores se sentirán desanimados por utilizarlas, pero sospecho que al final del libro habrán cambiado de opinión.

Cómo escribir anotaciones de tipos

Como hemos visto, para tipos internos como `int`, `bool` y `float` basta con utilizar el propio tipo como anotación. Pero ¿qué pasa si tenemos (por

ejemplo) un list?

```
def total(xs: list) -> float:
    return sum(xs)
```

Esto no es erróneo, pero el tipo no es lo bastante específico. Está claro que realmente queremos que `xs` sea un `list` de valores `float`, no (por ejemplo) un `list` de cadenas.

El módulo `typing` ofrece una serie de tipos parametrizados que podemos utilizar para hacer precisamente esto:

```
from typing import List      # observe la L mayúscula
def total(xs: List[float]) -> float:
    return sum(xs)
```

Hasta ahora hemos especificado solamente anotaciones para parámetros de función y tipos de retorno. Para las propias variables suele ser obvio cuál es el tipo:

```
# Así es como se anota el tipo de variables cuando se definen.
# Pero esto es innecesario; es "obvio" que x es un int.
x: int = 5
```

No obstante, algunas veces no es obvio:

```
values = []          # ¿cuál es mi tipo?
best_so_far = None    # ¿cuál es mi tipo?
```

En estos casos suministraremos las comprobaciones de tipos *inline*:

```
from typing import Optional
values: List[int] = []
best_so_far: Optional[float] = None    # permitido ser un float o None
```

El módulo `typing` contiene muchos otros tipos, de los que solo emplearemos unos pocos:

```
# las anotaciones de tipo de este fragmento son todas innecesarias
from typing import Dict, Iterable, Tuple
```

```
# las claves son strings, los valores son ints
counts: Dict[str, int] = {'data': 1, 'science': 2}
# las listas y los generadores son ambos iterables
if lazy:
    evens: Iterable[int] = (x for x in range(10) if x % 2 == 0)
else:
    evens = [0, 2, 4, 6, 8]
# las tuplas especifican un tipo para cada elemento
triple: Tuple[int, float, int] = (10, 2.3, 5)
```

Finalmente, como Python tiene funciones de primera clase, necesitamos un tipo que las represente también. Este es un ejemplo bastante forzado:

```
from typing import Callable
# La comprobación de tipos dice que el repetidor es una función que admite
# dos argumentos, un string y un int, y devuelve un string.
def twice(repeater: Callable[[str, int], str], s: str) -> str:
    return repeater(s, 2)
def comma_repeater(s: str, n: int) -> str:
    n_copies = [s for _ in range(n)]
    return ', '.join(n_copies)
assert twice(comma_repeater, "type hints") == "type hints, type hints"
```

Como las anotaciones de tipos son solo objetos Python, podemos asignarles variables para que sea más fácil hacer referencia a ellos:

```
Number = int
Numbers = List[Number]
def total(xs: Numbers) -> Number:
    return sum(xs)
```

Para cuando lleguemos al final del libro, el lector estará bastante familiarizado con leer y escribir anotaciones de tipos, y espero que las utilice en su código.

Bienvenido a DataSciencester

Esto concluye la orientación al empleado. Ah, otra cosa: intente no